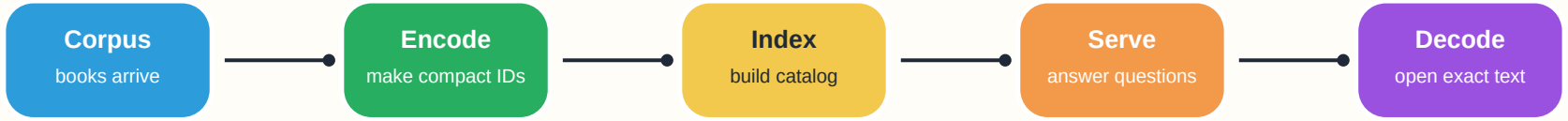
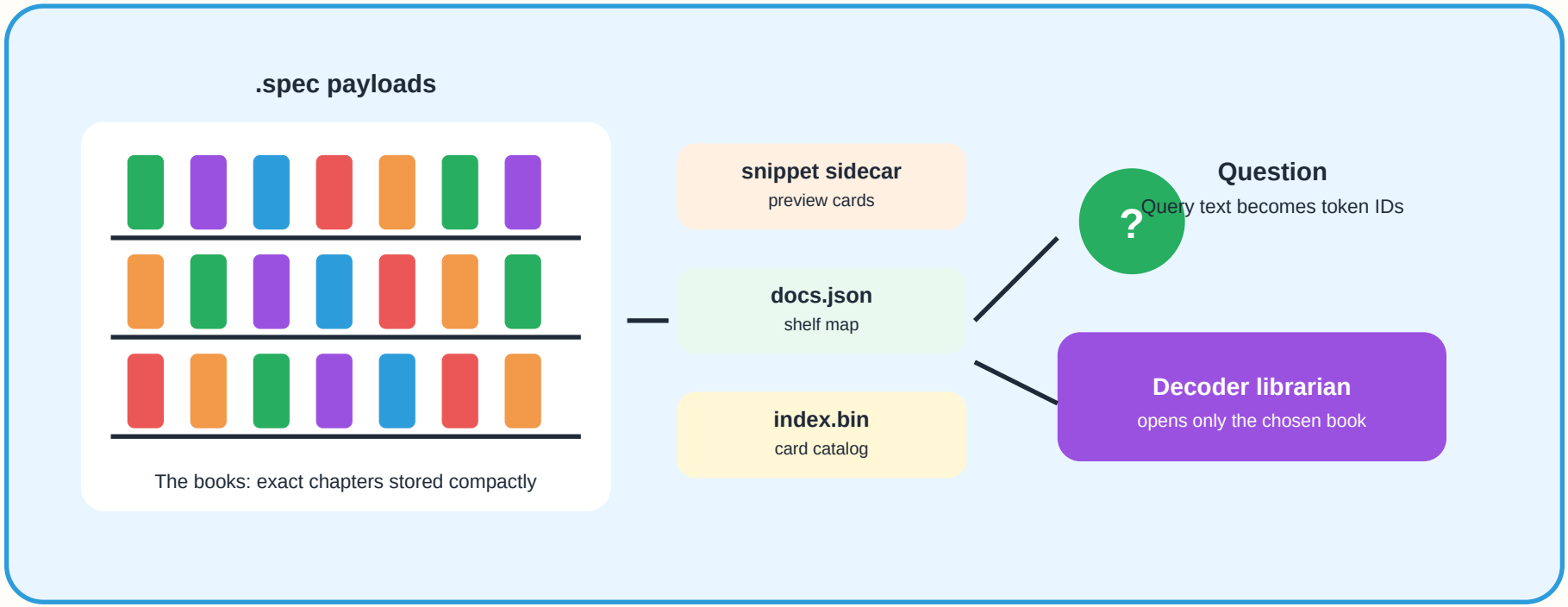
How Spectrum Stores, Serves, and Reads Files

A bright guide to the Spectrum process: from a corpus, to .spec storage, to fast search, to exact decoding.



Kid version	Engineer version
Spectrum is like a library that squeezes books onto tiny shelves, keeps a smart card catalog, and asks the librarian to open only the book you need.	Spectrum stores source text losslessly as .spec token streams, builds compact retrieval postings, serves snippets and metadata first, then hydrates selected payloads by decoding .spec bytes back to exact text.

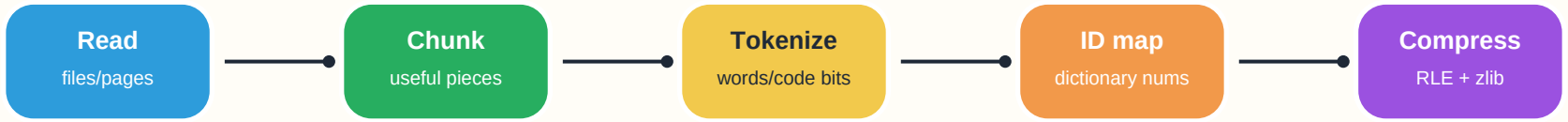
The Library Picture



Library thing	Spectrum thing	What it does
Book	.spec payload	Stores the original chapter exactly, but in compact token-ID form.
Shelf label	docs.json	Maps document IDs to titles, paths, chunk numbers, language IDs, and sizes.
Card catalog	index.bin / postings	Says which documents contain which meaningful tokens, so search can skip most books.
Bookmark card	snippet sidecar	Shows a small preview without opening and decoding the whole .spec file.
Librarian	decoder	Rebuilds the exact original text when a result is selected.

1. Encoding From a Corpus

A corpus is the pile of source material: code files, text documents, notes, or mixed project folders. Spectrum first chooses how to read that material, then turns the text into a stream of stable token IDs.



A .spec file is a labeled lunchbox

The first 16 bytes say how to read it. The body is compressed uint32 token IDs.

Step	What Spectrum uses	Where to look
Read corpus	Filesystem scanning, repo/demo inputs, JSONL records, or UTF-8 text directories.	CLI Tool/spectrum_cli/main.py, rag/storage_benchmark.py
Choose tokenizer	Extension/language mapping selects Python, JS, CSS, text, XML-compatible, Java, C/C++, Go, C#, shell, JSON/YAML/TOML, and more.	spec_format/spec_encoder.py, tokenizers/

Step	What Spectrum uses	Where to look
Map tokens to IDs	Known tokens become dictionary IDs. Unknown ASCII or Unicode characters use fallback IDs so the file stays lossless.	dictionary.py, english_tokens.py, spec_format/extension_tokens.py
Shrink runs	Repeated IDs can be encoded with an RLE marker. Then the ID stream is zlib-compressed.	spec_format/spec_encoder.py
Write .spec	Header stores magic bytes, dictionary version, flags, original length, language ID, and checksum. Body stores compressed uint32 IDs.	spec_format/spec_encoder.py

Important rule: the .spec payload must not throw away bytes. Retrieval tricks live beside it, not inside it.

2. Building the Search Catalog

After the books are stored, Spectrum builds the library catalog. It reads token IDs from the .spec files and keeps the useful retrieval tokens. It does not need to decode every file back to raw source text just to build the catalog.

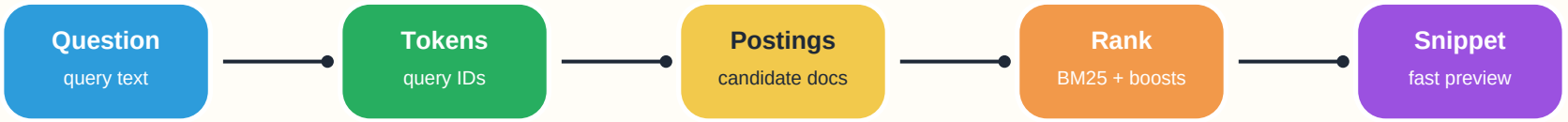
Catalog part	Job	Format
Document rows	One row per chunk/document: ID, path, title, language, original length, token count.	docs.json
Frequency vectors	For each document, count how often meaningful token IDs appear.	in-memory during build
Inverted postings	For each token ID, list the documents containing it, plus term frequency.	SPB1 or SPB2 binary index
Snippet sidecar	Small preview strings used for result lists so full decode can wait.	snippet_sidecar.json or built from chunks.jsonl

The newest compact postings path uses SPB2: document IDs are stored as gaps and numbers are encoded as variable-length integers. That makes the catalog smaller, like writing shelf numbers as short directions instead of full addresses every time.

- Search scoring uses BM25: rare useful words usually count more than words found everywhere.
- Candidate selection can drop very common tokens and focus on rare, path-like, or identifier-like tokens.
- Code reranking can add signals from paths, filenames, identifiers, function/class names, imports, and proximity.

3. Serving the Spectrum Store

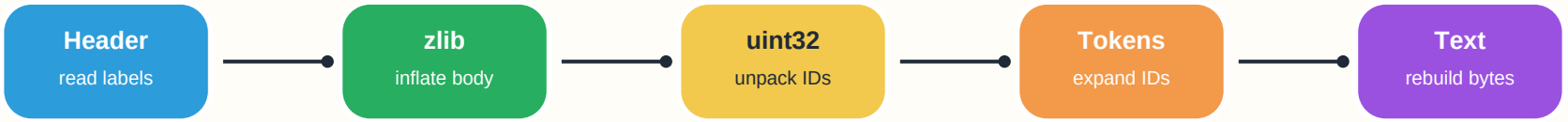
Serving is the part that answers questions quickly. The server loads metadata and the postings catalog, turns the user's question into Spectrum token IDs, finds likely documents, returns snippets, and only decodes the full payload when needed.



Serving action	What happens	Code path
Load store	If a .specpack exists, open it as a zip and read docs.json plus index.bin. Otherwise read a spectrum_spec directory.	rag/spectrum_serving.py
Preload or lazy-load .spec bytes	Payload bytes may be kept in RAM or read from the pack/file only when needed.	SpectrumServingRetriever.__init__
Search	Normalize query to token IDs, ask the binary BM25 postings index for candidates, then optionally rerank.	search_ids_with_trace
Return result list	Return title, source path, rank, and a windowed snippet. No full decode required for the list view.	search, windowed_snippet
Hydrate selected result	Decode selected .spec bytes and cache the decoded text for repeated access.	decode

4. Decoding Back to the Exact Text

Decoding is the librarian opening the chosen book and putting every word, space, and symbol back in the right place. It uses the header to know which dictionary version and language rules apply.



Decode step	Why it matters
Parse 16-byte header	Checks magic bytes, dictionary version, flags, original byte length, language ID, and checksum.
Choose token table	Current dictionary is used for current files; frozen old dictionaries allow older files to decode.
Decompress body	zlib turns compressed bytes back into a raw uint32 ID stream.
Expand special IDs	RLE markers repeat prior tokens; ASCII and Unicode fallback IDs become characters; dictionary IDs become stored token strings.
Reconstruct text	Plain text and XML-compatible payloads can use text reconstruction controls; code-like languages mostly join token strings.
Verify length/checksum	Confirms the decoded result matches the original byte length and checksum.

Outside Resources and Dependencies

Spectrum's core .spec encode/decode path is mostly local Python plus its own dictionary and tokenizer files. Some benchmark, corpus, runtime, and optional acceleration paths use outside libraries or outside data.

Area	Dependency/resource	Used for
Core encode/decode	Python standard library: struct, zlib, pathlib, sys, warnings	Binary headers, compressed token body, paths, version handling.
Dictionary	dictionary.py and generated english_tokens.py	Stable token IDs and large English word coverage.
Tokenizers	Local tokenizers/ package	Language-aware splitting for code, text, XML-compatible input, and config formats.
Extension compatibility	spec_format/extension_tokens.py	Preserves decoding compatibility for older extension-token payloads.
Corpus input	Local files, JSONL records, or cloned repositories	Builds benchmark/demo corpora without relying on a specific external corpus.
RAG benchmarks	numpy, scipy, scikit-learn	Conventional TF-IDF baseline and benchmark comparison store; not required for basic .spec decode.
Serving store	json, zipfile, re, dataclasses; local binary postings	Read .specpack zip bundles, docs metadata, snippets, and query features.
Optional native decode	Rust crate under native/spectrum_native	Faster byte-prism decoding path when available, with Python fallback.
CLI/runtime	Node/npm for CLI packaging and browser runtime files	spectrum command, JS decoder/service-worker experiments, and web serving shim.

Area	Dependency/resource	Used for
External demo inputs	User-provided repos or cloned GitHub repositories	Building demo corpora and .specpack benchmark outputs.

One Walk Through

Imagine a tiny library with three books: app.py, style.css, and notes.md.

- The encoder reads each book and picks the right reading glasses: Python tokenizer, CSS tokenizer, or text tokenizer.
- Words and code pieces become numbered library stickers from the Spectrum dictionary.
- Repeated stickers are shortened with RLE, then the sticker list is zipped with zlib and placed in .spec books.
- The catalog maker records which stickers appear in which books and writes index.bin plus docs.json.
- When a child asks, 'Where is the button color?', the librarian turns that question into stickers too.
- The catalog points to likely books quickly. The result list shows a bookmark snippet.
- Only when the child opens a result does the decoder rebuild the full original file exactly.

What stays true	What can change by profile
.spec payloads are meant to be lossless and portable. Dictionary IDs are append-only across versions, and older snapshots help decode older files.	Chunk sizes, overlap, BM25 settings, title/path boosts, sidecars, rerank depth, and corpus preprocessing can change per corpus profile.

Repo Map

File or folder	Why it matters
spec_format/spec_encoder.py	Writes .spec headers and compressed token ID bodies.
spec_format/spec_decoder.py	Reads .spec files and reconstructs exact text.
dictionary.py, english_tokens.py	Stable core dictionary and generated English token list.
tokenizers/	Language-aware tokenizers used before ID mapping.
rag/indexer.py	Builds retrieval indexes from .spec files without full source decode.
rag/storage_benchmark.py	Builds Spectrum stores, docs.json, binary postings, and benchmark baselines.
rag/spectrum_serving.py	Production-shaped serving retriever: load pack, search, snippets, decode selected payloads.
rag/query.py	Encodes queries and BM25 search over Spectrum indexes.
tools/build_hf_retrieval_corpus.py	Builds local retrieval corpora for storage and serving experiments.
CLI Tool/spectrum_cli/main.py	Public CLI encode/archive/index/search/demo commands.
Runtime/	Browser/service-worker and JS runtime experiments for serving .spec-backed web projects.
native/spectrum_native/	Optional Rust decoder acceleration path.

Generated from local repository context on 2026-05-09.